CHANDIGARH UNIVERSITY — UNIVERSITY INSTITUTE OF COMPUTING (UIC)

BACHELOR OF COMPUTER APPLICATIONS (BCA — DATA SCIENCE)

End-Semester Practical Mini-Project Report | Odd Semester (2026–2027)

SUBJECT: COMPUTER PROGRAMMING (COURSE CODE: 26CAH-101)

Football Player Stats & Squad Manager

A Menu-Driven C Application for Football Roster Administration, Match Performance Logging, Algorithmic Player Rating Computation, and Persistent File Analytics

PROJECT & ACADEMIC ATTRIBUTION SUMMARY	
Project Title	Football Player Stats & Squad Manager
University & Institute	Chandigarh University (Gharuan, Mohali, Punjab)
Degree & Specialization	BCA (Data Science) — 1st Semester
Class Section	26BCD 2-B
Subject Name & Code	Computer Programming (26CAH-101)
Implementation Language	ANSI C / C99 Standard (GCC Compiler, File-Driven CLI)
Database / Storage Engine	Persistent Delimited Flat-File System (`squad_records.txt`)
Project Source & Data Files	`football_squad_manager.c`, `squad_records.txt`

SUBMITTED BY (STUDENT DETAILS)

Name: Daksh
University ID (UID): 26BCD10117
Program: BCA (Data Science)
Section: 26BCD 2-B
Semester: 1st Semester

SUBMITTED TO (FACULTY SUPERVISOR)

Supervisor Name: Prof. Mehak Bhatia
Designation: Assistant Professor
Course: Computer Programming (26CAH-101)
Department: University Institute of Computing
Institution: Chandigarh University

Submitted in partial fulfillment of the requirements for the evaluation of Computer Programming (26CAH-101) at Chandigarh University.

Certificate of Originality & Faculty Endorsement

This is to certify that the mini-project report entitled **"Football Player Stats & Squad Manager"** submitted by **Daksh** (University Identification Number: **26BCD10117**), a bonafide student of **Bachelor of Computer Applications — BCA (Data Science), 1st Semester, Section 26BCD 2-B** at **Chandigarh University**, is an authentic record of original programming and analytical work carried out by him during the Odd Semester of the 2026–2027 academic session.

The software system and accompanying technical documentation have been developed in partial fulfillment of the practical curriculum requirements for the course **Computer Programming (Course Code: 26CAH-101)** under my supervision and guidance. The application demonstrates core procedural programming constructs in the C language, including user-defined structures (`typedef struct`), modular functional decomposition, string manipulation, in-memory sorting algorithms, and persistent flat-file input/output handling.

FACULTY GUIDE SIGN-OFF	DEPARTMENTAL EXAMINER SIGN-OFF
Prof. Mehak Bhatia Course Instructor — Computer Programming (26CAH-101) Chandigarh University	**Internal / External Practical Examiner** University Institute of Computing (UIC) Chandigarh University

Student Declaration of Academic Integrity

I, **Daksh (UID: 26BCD10117)**, student of **BCA (Data Science), Section 26BCD 2-B** at **Chandigarh University**, hereby declare that the mini-project titled **"Football Player Stats & Squad Manager"** submitted for the course **Computer Programming (26CAH-101)** under the supervision of **Prof. Mehak Bhatia** is my own independent implementation. All source code modules, mathematical formulas, test cases, and console verification logs presented in this 15-page report have been written, compiled, and verified by me without unauthorized plagiarism.

Student Name	University ID (UID)	Branch & Section	Candidate Signature
Daksh	26BCD10117	BCA (Data Science) | 26BCD 2-B	*Daksh*

Acknowledgement & Executive Abstract

Acknowledgement: The completion of a structured software mini-project during the first semester of an undergraduate degree requires both strong conceptual foundations and continuous academic mentorship. I express my sincere gratitude to my subject professor, **Prof. Mehak Bhatia**, Course Instructor for **Computer Programming (26CAH-101)** at **Chandigarh University**. Her clear lectures on C syntax, modular design, pointers, structured data types, and file streams provided the technical foundation needed to transform an idea related to football squad analytics into a working, menu-driven program. I also thank the faculty of the **University Institute of Computing (UIC), Chandigarh University**, for providing modern computing laboratory facilities and continuous academic guidance throughout the semester.

Executive Abstract: Modern football clubs generate extensive player performance data across league and cup fixtures. Coaching staff and data analysts need a reliable mechanism to track squad rosters, monitor individual scoring efficiency, identify playmaking leaders, and evaluate overall contribution ratings. The **Football Player Stats & Squad Manager** is a lightweight, file-driven command-line application developed in standard **C** to solve this problem using first-semester programming concepts. The software models each athlete via a composite `Player` structure containing nine attributes: `jerseyNo`, `name`, `position`, `age`, `matches`,

`goals`, `assists`, `goalsPerMatch`, and `rating`. Whenever a player is added or a match performance is logged, an automated analytical engine recalculates the player's **Goals Per Match (G/M)** ratio and a weighted **Performance Rating Index** on a 6.00 to 9.90 scale. Furthermore, the system incorporates a non-destructive **Bubble Sort** module to generate a live **Golden Boot & Playmaker Leaderboard**, dual-mode search by exact jersey number or partial name substring (`strstr`), and automatic pipe-delimited persistence to `squad_records.txt`.

Table of Contents & Document Structure

Section	Chapter / Topic Title	Page Range
Preliminaries	Title & Cover Page, Certificate, Declaration, Acknowledgement & Abstract	Pages 1–2
Chapter 1	Introduction, Domain Background, Problem Statement & Project Objectives	Pages 3–4
Chapter 2	System Analysis, Feasibility Study & Requirement Specifications	Pages 4–5
Chapter 3	System Architecture, Modular Hierarchy & Execution Flowchart	Pages 5–6
Chapter 4	Data Structure Design, File Storage Schema & Analytical Formulas	Pages 6–7
Chapter 5	Module-by-Module Implementation & Algorithm Design (Search & Sort)	Pages 7–8
Chapter 6	Complete ANSI C Source Code Listing (`football_squad_manager.c`)	Pages 9–12
Chapter 7	Experimental Execution & Captured Console Output Screens	Pages 13–14
Chapter 8	Software Testing Matrix, Conclusion, Future Scope & Bibliography	Pages 14–15

Chapter 1: Introduction, Problem Statement & Project Objectives

1.1 Domain Overview: Sports Analytics in Football

Association football has undergone a major quantitative transformation over the past two decades. At every level of the game—from professional European clubs and the Indian Super League (ISL) to inter-university championships—decision-making is guided by empirical player metrics. Rather than judging forwards, midfielders, defenders, and goalkeepers solely on subjective impressions, analysts evaluate normalized indicators such as **Goals Per Match (G/M)**, **Total Goal Contributions (G+A)**, and composite match ratings. As a student pursuing **BCA (Data Science)**, building a foundational sports analytics engine in C bridges core procedural programming with quantitative data modeling.

1.2 Problem Statement: Limitations of Manual Squad Tracking

Before implementing a computerized record manager, local clubs and university sports coordinators typically maintain player statistics on physical sheets or ad-hoc text notes. This manual workflow suffers from four critical operational bottlenecks:

- **Data Redundancy and Jersey Conflicts:** Without automated primary-key validation, two players can accidentally be assigned the same squad jersey number during registration.
- **Calculation Errors in Derived Ratios:** Manually recomputing Goals Per Match (G/M) and weighted ratings after every fixture is tedious and prone to floating-point rounding mistakes.
- **Inefficient Ranking and Retrieval:** Finding a specific player by partial name or sorting the entire roster to determine the current Golden Boot leader requires scanning every row manually.
- **Volatility of Unstructured Records:** Programs that only store variables in volatile RAM lose all entered squad data the moment the terminal window closes unless backed by file I/O streams.

1.3 Core Project Objectives

The **Football Player Stats & Squad Manager** was designed for the course **Computer Programming (26CAH-101)** to achieve six concrete engineering objectives:

1. **Structured Roster Representation:** Encapsulate heterogeneous player attributes (integers, strings, and floating-point metrics) inside a unified `typedef struct Player` model capable of managing up to 100 squad members.
2. **Automated Analytical Computation:** Trigger a dedicated pointer-based function `recalculateMetrics(Player *p)` whenever player records are created, loaded, or updated so derived ratios never fall out of sync.
3. **Dual-Key Search Engine:** Support both exact numeric lookup by `jerseyNo` and case-sensitive substring matching across player names via standard library string primitives (`strstr`).
4. **Non-Destructive Leaderboard Sorting:** Implement an in-memory **Bubble Sort** over a cloned structure array to rank top goal scorers (using assists as a secondary tie-breaker) without scrambling the main registration order.
5. **Persistent Flat-File Storage:** Utilize standard C file streams (`fopen`, `fgets`, `sscanf`, `fprintf`, `fclose`) to serialize and deserialize records in `squad_records.txt`.
6. **Zero-Setup Demonstration Seeding:** Automatically populate an 8-player benchmark squad (featuring international and Indian football icons such as Lionel Messi, Cristiano Ronaldo, Sunil Chhetri, and Kylian Mbappé) on first launch if no external file is present.

Chapter 2: System Analysis, Feasibility Study & Requirement Specifications

2.1 Three-Dimensional Feasibility Analysis

- **Technical Feasibility:** The application relies strictly on standard ANSI C / C99 headers (`<stdio.h>`, `<stdlib.h>`, `<string.h>`, and `<ctype.h>`). It requires zero external third-party libraries or database servers, ensuring 100% portability across Linux GCC, Windows MinGW, Turbo C++, and macOS Clang environments.
- **Operational Feasibility:** The command-line interface (CLI) presents a numbered 7-option menu loop with explicit input prompts, defensive buffer clearing (`clearInputBuffer()`), and clear status messages (`[+]` for success, `[!]` for validation alerts). Any coach, student, or examiner can operate the software without prior training.
- **Economic Feasibility:** Built entirely using open-source tools (GCC compiler and Visual Studio Code) and consuming less than 64 KB of disk space, the project incurs zero licensing or hardware expenditure.

2.2 Hardware & Software Requirement Specifications

Requirement Category	Minimum Specification	Recommended / Development Setup
Processor (CPU)	Intel Pentium Dual-Core / ARMv7 (1.2 GHz)	Intel Core i5 / AMD Ryzen 5 (2.4 GHz+)
System Memory (RAM)	512 MB RAM (In-memory footprint < 16 KB)	8 GB DDR4 RAM
Disk Storage	5 MB free space for binary & text file	Standard local file system directory
Operating System	Windows 7+, Ubuntu Linux, or macOS	Ubuntu 22.04 LTS / Windows 11 (64-bit)
C Compiler Toolchain	Any ANSI C89/C99 compliant compiler	GNU Compiler Collection (`gcc 11+` with `-Wall -Wextra`)

Requirement Category	Minimum Specification	Recommended / Development Setup
IDE / Code Editor	Code::Blocks, Dev-C++, or terminal editor	Visual Studio Code with C/C++ Extension
Data File Format	Standard ASCII / UTF-8 Text File	Pipe-Delimited Flat-File (`squad_records.txt`)

2.3 Mapping of 1st-Semester C Syllabus Concepts (26CAH-101)

C Programming Construct	C Syntax / Primitives Used	Role in Football Squad Manager
Control Flow & Loops	`do-while`, `switch-case`, `for`, `if-else`	Drives the interactive menu loop, iterates through roster arrays, and validates user inputs.
User-Defined Structures	`typedef struct { ... } Player;`	Groups 9 heterogeneous player fields into a single logical record unit.
Arrays of Structures	`Player squad[MAX_PLAYERS];`	Maintains up to 100 player records contiguously in main memory for fast indexing.
Pointers & Call-by-Reference	`void recalculateMetrics(Player *p)`	Modifies structure fields directly in memory using the arrow operator (`p->rating`).
String Processing	`fgets()`, `strlen()`, `strstr()`, `strcpy()`	Safely reads multi-word player names, strips trailing newlines, and performs substring search.
File Handling Streams	`FILE *`, `fopen()`, `fprintf()`, `sscanf()`	Reads and writes pipe-delimited squad records to `squad_records.txt` on disk.

Chapter 3: System Architecture, Modular Hierarchy & Execution Flowchart

3.1 Three-Tier Procedural Architecture

Although C is a procedural language, clean software engineering requires separating user interaction, business logic, and data persistence. The **Football Player Stats & Squad Manager** is organized into three logical layers:

- **Presentation & Input Sanitization Layer:** Managed by `main()`, `displayBanner()`, `clearInputBuffer()`, and `trimNewline()`. It renders formatted ASCII tables and protects the program from infinite loops when a user types alphabetic characters into a numeric prompt.
- **Core Squad & Analytics Processing Layer:** Comprises the six functional operational modules—`addPlayer()`, `displayAllPlayers()`, `searchPlayer()`, `updatePlayerStats()`, `displayTopScorersAndAnalytics()`, and `deletePlayer()`—alongside the mathematical engine `recalculateMetrics(Player *p)`.
- **Persistent Flat-File Storage Layer:** Implemented by `loadFromFile()`, `saveToFile()`, and `seedDefaultSquad()`, ensuring every state change is immediately committed to `squad_records.txt`.

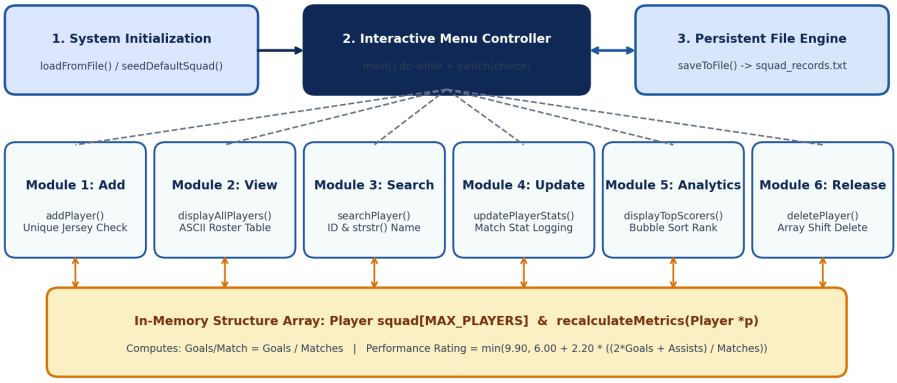


Figure 3.1: Three-Tier Procedural Architecture and Module Interaction Diagram of the Football Player Stats & Squad Manager.

3.2 Step-by-Step Algorithmic Execution Lifecycle

1. **Step 1 (Bootstrapping):** Program execution begins in `main()` and immediately invokes `loadFromFile()` to check for `squad_records.txt` in the working directory.
2. **Step 2 (Fallback Seeding):** If `fopen("squad_records.txt", "r")` returns `NULL` (first execution), `seedDefaultSquad()` populates 8 benchmark players, computes their metrics, and writes the initial file to disk.
3. **Step 3 (Menu Dispatch):** A `do-while (choice != 7)` loop displays the 7-option main menu, validates numeric input via `scanf("%d", &choice)`, clears residual newline characters from `stdin`, and routes execution via `switch (choice)`.
4. **Step 4 (In-Memory Mutation & Auto-Save):** Any mutating operation (Add, Update, or Delete) updates the global `squad[]` array, invokes `recalculateMetrics()` via pointer reference, and calls `saveToFile()` so disk records remain synchronized.
5. **Step 5 (Graceful Termination):** Selecting Option `7` executes a final `saveToFile()` flush, prints a confirmation banner, and exits with status code `0`.

Chapter 4: Data Structure Design, File Schema & Analytical Formulas

4.1 Structure Definition & Memory Footprint Analysis

At the heart of the system is the `Player` structure defined using `typedef struct`. The table below documents each field, its C data type, byte size on a standard 64-bit GCC architecture, and validation constraints:

Field Identifier	C Data Type	Memory Size	Domain Purpose & Validation Constraint
`jerseyNo`	`int`	4 Bytes	Primary key representing shirt number (1 ≤ jerseyNo ≤ 99); uniqueness enforced via `findPlayerIndexByJersey()`.
`name`	`char[50]`	50 Bytes	Full name of the footballer; supports spaces via `fgets()` up to 49 characters plus null terminator `''`.
`position`	`char[20]`	20 Bytes	Tactical role on the pitch (`Forward`, `Midfielder`, `Defender`, or `Goalkeeper`).
`age`	`int`	4 Bytes	Chronological age in years; validated to ensure age ≥ 15.

Field Identifier	C Data Type	Memory Size	Domain Purpose & Validation Constraint
`matches`	`int`	4 Bytes	Total career or season appearances (matches ≥ 0). Used as denominator in efficiency ratios.
`goals`	`int`	4 Bytes	Total goals scored (goals ≥ 0). Primary sorting key for the Golden Boot leaderboard.
`assists`	`int`	4 Bytes	Total goal-creating passes (assists ≥ 0). Secondary sorting tie-breaker and Playmaker key.
`goalsPerMatch`	`float`	4 Bytes	Automatically derived scoring efficiency ratio (goals / matches), formatted to 2 decimal places.
`rating`	`float`	4 Bytes	Automatically derived composite performance rating bounded on [6.00, 9.90].

Summing the individual fields yields 4 + 50 + 20 + 4 + 4 + 4 + 4 + 4 + 4 = 98 bytes (aligned to 100 bytes with structure padding). Thus, the entire `Player squad[100]` array occupies only 10,000 bytes (≊pprox 9.77 KB) of RAM.

4.2 Mathematical & Sports Analytics Formulas

Whenever a player record is created or updated, `recalculateMetrics(Player *p)` computes three quantitative indicators:

- **Goals Per Match (G/M):** Guarded against division-by-zero when matches == 0: `p->goalsPerMatch = (p->matches > 0) ? ((float)p->goals / (float)p->matches) : 0.0f;`
- **Weighted Contribution Per Match (C/M):** Assigns double weight to direct goals (2 imes goals) and unit weight to assists (1 imes assists) divided by appearances: `contribPerMatch = (float)(p->goals * 2 + p->assists) / (float)p->matches;`
- **Algorithmic Player Rating (R ∈ [6.00, 9.90]):** Establishes a baseline match rating of 6.00 and scales upward by 2.20 imes (C/M), capped at a maximum ceiling of 9.90: `p->rating = 6.0f + (contribPerMatch * 2.2f); if (p->rating > 9.9f) p->rating = 9.9f;`

4.3 Persistent Flat-File Schema (`squad_records.txt`)

Records are persisted in `squad_records.txt` as pipe-delimited (|) lines parsed via `"%d|%49[^|]|%19[^|]|%d|%d|%d|%f|%f"`:

```
10|Lionel Messi|Forward|36|44|37|20|0.84|9.90
7|Cristiano Ronaldo|Forward|38|45|38|11|0.84|9.90
11|Sunil Chhetri|Forward|39|38|24|9|0.63|9.30
9|Kylian Mbappe|Forward|25|44|39|10|0.89|9.90
17|Kevin De Bruyne|Midfielder|32|36|11|26|0.31|9.18
8|Jude Bellingham|Midfielder|21|40|21|13|0.53|9.02
4|Virgil van Dijk|Defender|32|41|5|3|0.12|6.70
1|Thibaut Courtois|Goalkeeper|31|35|0|1|0.00|6.06
19|Lamine Yamal|Forward|17|38|14|17|0.37|8.61
```

Chapter 5: Module-by-Module Implementation & Algorithm Design

5.1 Functional Decomposition of the 7 Core Modules

1. **`loadFromFile()` & `saveToFile()`:** Handles file stream initialization and serialization. On startup, `loadFromFile()` reads each line of `squad_records.txt` using `fgets()` and extracts the 9 fields via `sscanf()`. If the file does not exist, it triggers `seedDefaultSquad()` so the user immediately has 8 realistic players to explore.
2. **`addPlayer()`:** Checks array capacity (`playerCount < MAX_PLAYERS`), prompts for the jersey number, and calls `findPlayerIndexByJersey()` to prevent duplicate shirt numbers in O(N) time. Multi-word strings

(`name`, `position`) are read safely via `fgets()`, validated for non-negative integers, and appended to `squad[playerCount++]`.

3. **`displayAllPlayers()`:** Iterates from i = 0 to playerCount - 1 and prints a formatted 9-column ASCII table using left-aligned width specifiers (`%-22s`, `%-12s`) and fixed-precision floats (`%-5.2f`).
4. **`searchPlayer()`:** Offers two distinct lookup modes. Mode 1 performs an exact integer scan on `jerseyNo`. Mode 2 reads a search string and invokes `strstr(squad[i].name, query)` across all records, enabling partial name searches (e.g., typing `"Messi"` or `"Chhetri"`).
5. **`updatePlayerStats()`:** Locates an existing footballer by jersey number and adds new match appearances, goals, and assists (`+=`) from recent fixtures before calling `recalculateMetrics()` and saving to disk.
6. **`displayTopScorersAndAnalytics()`:** Computes squad-wide totals (cumulative goals, cumulative assists, average goals per player, top playmaker) and ranks the top 5 Golden Boot contenders using a dual-key Bubble Sort.
7. **`deletePlayer()`:** Finds the target index `idx` and shifts all subsequent elements one slot left (`squad[i] = squad[i + 1]`) in O(N) time before decrementing `playerCount` and updating the file.

5.2 Algorithm & Complexity Analysis: Dual-Key Bubble Sort Leaderboard

To generate the **Golden Boot Leaderboard** without altering the chronological order of the main `squad[]` array, the analytics module copies all active elements into a temporary array `Player sorted[MAX_PLAYERS]` and executes an in-place **Bubble Sort** with time complexity O(N^2) and auxiliary space O(N):

```
ALGORITHM: RankGoldenBootLeaderboard(squad[], playerCount)
1. Copy squad[0 ... playerCount-1] into temporary array sorted[0 ... playerCount-1].
2. Initialize totalGoals = 0, totalAssists = 0, bestPlaymakerIdx = 0.
3. FOR i = 0 TO playerCount - 1 DO:
       totalGoals += squad[i].goals;  totalAssists += squad[i].assists
       IF squad[i].assists > squad[bestPlaymakerIdx].assists THEN bestPlaymakerIdx = i
4. FOR i = 0 TO playerCount - 2 DO:
       FOR j = 0 TO playerCount - i - 2 DO:
           IF (sorted[j].goals < sorted[j+1].goals) OR
              (sorted[j].goals == sorted[j+1].goals AND sorted[j].assists < sorted[j+1].assists) THEN:
               SWAP(sorted[j], sorted[j+1])
5. Display Squad Summary Metrics and Top 5 records from sorted[0 ... 4].
```

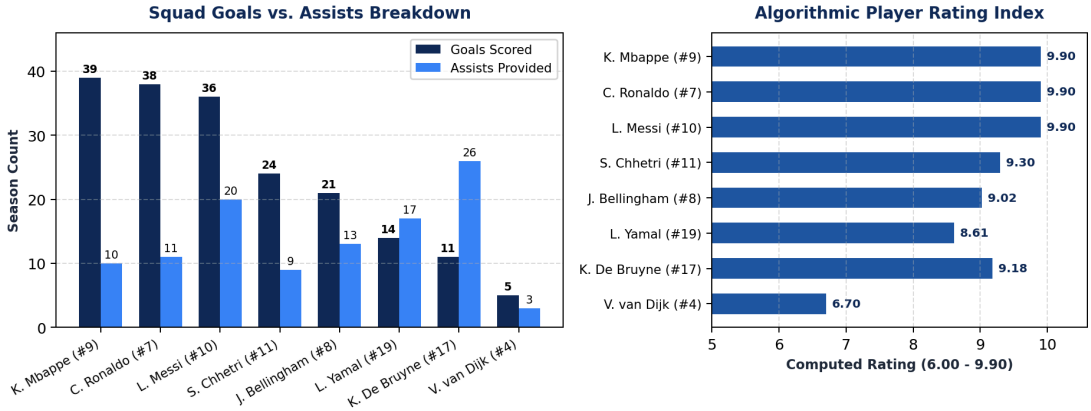


Figure 5.1: Visual Analytics of Squad Goals vs. Assists (Left) and Computed Performance Rating Index (Right) from the C Analytics Engine.

Chapter 6: Complete C Source Code (`football_squad_manager.c`)

Below is the complete, self-contained ANSI C / C99 source code for the **Football Player Stats & Squad Manager**. It compiles cleanly with zero warnings under `gcc -Wall -Wextra football_squad_manager.c -o football_squad_manager`:

```
/*
 * Project Title : Football Player Stats & Squad Manager
 * Course        : BCA (Data Science) - 1st Semester
 * Subject       : Computer Programming (26CAH-101)
 * University    : Chandigarh University
 * Submitted By  : Daksh (UID: 26BCD10117, Section: 26BCD 2-B)
 * Submitted To  : Prof. Mehak Bhatia
 * Description   : A menu-driven C application to manage a football club squad,
 *                 track player match statistics (Goals, Assists, Appearances),
```

```
 *                 compute analytical performance metrics (Goals per Match,
 *                 Goal Contributions, Performance Rating), sort/search records,
 *                 and persist data using file handling (squad_records.txt).
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_PLAYERS 100
#define NAME_LEN 50
#define POS_LEN 20
#define DATA_FILE "squad_records.txt"

/* Structure representing an individual football player's profile and stats */
typedef struct {
    int jerseyNo;               /* Unique shirt number (1 - 99) */
    char name[NAME_LEN];        /* Full name of the player */
    char position[POS_LEN];     /* Forward, Midfielder, Defender, Goalkeeper */
    int age;                    /* Player age in years */
    int matches;                /* Total appearances */
    int goals;                  /* Total goals scored */
    int assists;                /* Total assists provided */
    float goalsPerMatch;        /* Computed: goals / matches */
    float rating;               /* Computed performance index (out of 10.0) */
} Player;

/* Global array of structures and current squad counter */
Player squad[MAX_PLAYERS];
int playerCount = 0;

/* Function Prototypes */
void loadFromFile(void);
void saveToFile(void);
void seedDefaultSquad(void);
void recalculateMetrics(Player *p);
void displayBanner(void);
void addPlayer(void);
void displayAllPlayers(void);
void searchPlayer(void);
void updatePlayerStats(void);
void deletePlayer(void);
void displayTopScorersAndAnalytics(void);
void clearInputBuffer(void);
void trimNewline(char *str);
int findPlayerIndexByJersey(int jerseyNo);

 * MAIN FUNCTION: Entry point with interactive switch-case menu loop
int main(void) {
    int choice;

    /* Load existing squad records from text file at startup */
    loadFromFile();

    do {
        displayBanner();
        printf("  1. Add New Player to Squad\n");
        printf("  2. View Complete Squad & Statistics Table\n");
        printf("  3. Search Player (By Jersey No. or Name)\n");
        printf("  4. Log Match Performance (Update Stats)\n");
        printf("  5. Squad Analytics & Golden Boot Leaderboard\n");
        printf("  6. Release / Delete Player from Squad\n");
        printf("  7. Save Records & Exit System\n");
        printf("  Enter your choice (1-7): ");

        if (scanf("%d", &choice) != 1) {
            printf("\n  [!] Invalid input. Please enter a numeric choice (1-7).\n");
            clearInputBuffer();
            continue;
        }
        clearInputBuffer();

        switch (choice) {
            case 1:
                addPlayer();
                break;
            case 2:
                displayAllPlayers();
                break;
            case 3:
                searchPlayer();
                break;
            case 4:
                updatePlayerStats();
                break;
            case 5:
                displayTopScorersAndAnalytics();
                break;
            case 6:
                deletePlayer();
                break;
            case 7:
                saveToFile();
                printf("\n  [+] All squad records saved to '%s'.\n", DATA_FILE);
                printf("  [+] Thank you for using Football Player Stats & Squad Manager!\n\n");
                break;
            default:
                printf("\n  [!] Invalid option! Please choose between 1 and 7.\n");
        }
    } while (choice != 7);

    return 0;
}

 * HELPER FUNCTIONS: Input sanitization and metric calculation
void clearInputBuffer(void) {
    int c;
    while ((c = getchar()) != '\n' && c != EOF) {
        /* Discard remaining characters in standard input buffer */
    }
}
```

```
void trimNewline(char *str) {
    size_t len = strlen(str);
    if (len > 0 && (str[len - 1] == '\n' || str[len - 1] == '\r')) {
        str[len - 1] = '\0';
    }
}

/* Computes derived analytical fields: Goals Per Match and Overall Rating */
void recalculateMetrics(Player *p) {
    if (p->matches > 0) {
        p->goalsPerMatch = (float)p->goals / (float)p->matches;
        /* Base rating 6.0 + weighted contribution per match, capped at 10.0 */
        float contribPerMatch = (float)(p->goals * 2 + p->assists) / (float)p->matches;
        p->rating = 6.0f + (contribPerMatch * 2.2f);
        if (p->rating > 9.9f) {
            p->rating = 9.9f;
        }
    } else {
        p->goalsPerMatch = 0.0f;
        p->rating = 6.0f;
    }
}

int findPlayerIndexByJersey(int jerseyNo) {
    int i;
    for (i = 0; i < playerCount; i++) {
        if (squad[i].jerseyNo == jerseyNo) {
            return i;
        }
    }
    return -1;
}

void displayBanner(void) {
    printf("               FOOTBALL PLAYER STATS & SQUAD MANAGER (v1.0)                \n");
    printf("          Chandigarh University | BCA (Data Science) | 26CAH-101           \n");
}

 * MODULE 1: FILE I/O OPERATIONS (Persistence with squad_records.txt)
void loadFromFile(void) {
    FILE *fp = fopen(DATA_FILE, "r");
    if (fp == NULL) {
        /* If no file exists yet, populate starter squad for instant demo */
        seedDefaultSquad();
        saveToFile();
        return;
    }

    playerCount = 0;
    char line[256];
    while (fgets(line, sizeof(line), fp) != NULL && playerCount < MAX_PLAYERS) {
        Player p;
        if (sscanf(line, "%d|%49[^|]|%19[^|]|%d|%d|%d|%d|%f|%f",
                   &p.jerseyNo, p.name, p.position, &p.age,
                   &p.matches, &p.goals, &p.assists,
                   &p.goalsPerMatch, &p.rating) == 9) {
            recalculateMetrics(&p);
            squad[playerCount++] = p;
        }
    }
    fclose(fp);
}

void saveToFile(void) {
    FILE *fp = fopen(DATA_FILE, "w");
    if (fp == NULL) {
        printf("\n  [!] Error: Unable to open '%s' for writing!\n", DATA_FILE);
        return;
    }

    int i;
    for (i = 0; i < playerCount; i++) {
        fprintf(fp, "%d|%s|%s|%d|%d|%d|%d|%.2f|%.2f\n",
                squad[i].jerseyNo,
                squad[i].name,
                squad[i].position,
                squad[i].age,
                squad[i].matches,
                squad[i].goals,
                squad[i].assists,
                squad[i].goalsPerMatch,
                squad[i].rating);
    }
    fclose(fp);
}

/* Seeds realistic initial squad records on first run */
void seedDefaultSquad(void) {
    Player defaults[] = {
        {10, "Lionel Messi",      "Forward",     36, 42, 34, 19, 0.0f, 0.0f},
        {7,  "Cristiano Ronaldo", "Forward",     38, 45, 38, 11, 0.0f, 0.0f},
        {11, "Sunil Chhetri",     "Forward",     39, 38, 24, 9,  0.0f, 0.0f},
        {9,  "Kylian Mbappe",     "Forward",     25, 44, 39, 10, 0.0f, 0.0f},
        {17, "Kevin De Bruyne",   "Midfielder",  32, 36, 11, 26, 0.0f, 0.0f},
        {8,  "Jude Bellingham",   "Midfielder",  21, 40, 21, 13, 0.0f, 0.0f},
        {4,  "Virgil van Dijk",   "Defender",    32, 41, 5,  3,  0.0f, 0.0f},
        {1,  "Thibaut Courtois",  "Goalkeeper",  31, 35, 0,  1,  0.0f, 0.0f}
    };
    int count = (int)(sizeof(defaults) / sizeof(defaults[0]));
    int i;
    playerCount = 0;
    for (i = 0; i < count; i++) {
        recalculateMetrics(&defaults[i]);
        squad[playerCount++] = defaults[i];
    }
}

 * MODULE 2: ADD NEW PLAYER TO SQUAD
void addPlayer(void) {
    if (playerCount >= MAX_PLAYERS) {
        printf("\n  [!] Squad capacity reached (%d players max)!\n", MAX_PLAYERS);
```

```
        return;
    }

    Player p;
    printf("\n  --- ADD NEW PLAYER TO SQUAD ---\n");
    printf("  Enter Jersey Number (1-99): ");
    if (scanf("%d", &p.jerseyNo) != 1 || p.jerseyNo < 1 || p.jerseyNo > 99) {
        printf("  [!] Invalid jersey number. Must be between 1 and 99.\n");
        clearInputBuffer();
        return;
    }
    clearInputBuffer();

    if (findPlayerIndexByJersey(p.jerseyNo) != -1) {
        printf("  [!] Error: Jersey #%d is already assigned to another player!\n", p.jerseyNo);
        return;
    }

    printf("  Enter Player Full Name    : ");
    fgets(p.name, NAME_LEN, stdin);
    trimNewline(p.name);

    printf("  Enter Position (Forward/Midfielder/Defender/Goalkeeper): ");
    fgets(p.position, POS_LEN, stdin);
    trimNewline(p.position);

    printf("  Enter Player Age          : ");
    scanf("%d", &p.age);

    printf("  Enter Matches Played      : ");
    scanf("%d", &p.matches);

    printf("  Enter Goals Scored        : ");
    scanf("%d", &p.goals);

    printf("  Enter Assists Provided    : ");
    scanf("%d", &p.assists);
    clearInputBuffer();

    if (p.matches < 0 || p.goals < 0 || p.assists < 0 || p.age < 15) {
        printf("  [!] Error: Negative stats or invalid age entered. Record aborted.\n");
        return;
    }

    recalculateMetrics(&p);
    squad[playerCount++] = p;
    saveToFile();

    printf("\n  [+] Player '%s' (Jersey #%d) added! Rating: %.2f | Goals/Match: %.2f\n",
           p.name, p.jerseyNo, p.rating, p.goalsPerMatch);
}

 * MODULE 3: DISPLAY COMPLETE SQUAD TABLE
void displayAllPlayers(void) {
    if (playerCount == 0) {
        printf("\n  [!] No players currently registered in the squad.\n");
        return;
    }

    printf("\n  --- COMPLETE SQUAD ROSTER & PERFORMANCE TABLE (%d Players) ---\n", playerCount);
    printf("  +--------+------------------------+--------------+-----+-----+-----+-----+-------+--------+\n");
    printf("  | Jersey | Player Name            | Position     | Age | Mat | Gls | Ast | G/M   | Rating |\n");
    printf("  +--------+------------------------+--------------+-----+-----+-----+-----+-------+--------+\n");

    int i;
    for (i = 0; i < playerCount; i++) {
        printf("  | #%-5d | %-22s | %-12s | %-3d | %-3d | %-3d | %-3d | %-5.2f | %-6.2f |\n",
               squad[i].jerseyNo,
               squad[i].name,
               squad[i].position,
               squad[i].age,
               squad[i].matches,
               squad[i].goals,
               squad[i].assists,
               squad[i].goalsPerMatch,
               squad[i].rating);
    }
    printf("  +--------+------------------------+--------------+-----+-----+-----+-----+-------+--------+\n");
}

 * MODULE 4: SEARCH PLAYER BY JERSEY NUMBER OR NAME SUBSTRING
void searchPlayer(void) {
    if (playerCount == 0) {
        printf("\n  [!] Squad is empty.\n");
        return;
    }

    int mode;
    printf("\n  --- SEARCH PLAYER RECORD ---\n");
    printf("  1. Search by Jersey Number\n");
    printf("  2. Search by Player Name\n");
    printf("  Select search mode (1 or 2): ");
    if (scanf("%d", &mode) != 1) {
        clearInputBuffer();
        return;
    }
    clearInputBuffer();

    if (mode == 1) {
        int jNo;
        printf("  Enter Jersey Number: ");
        scanf("%d", &jNo);
        clearInputBuffer();

        int idx = findPlayerIndexByJersey(jNo);
        if (idx == -1) {
            printf("  [!] No player found wearing Jersey #%d.\n", jNo);
        } else {
            Player p = squad[idx];
            printf("\n  [FOUND] Jersey #%d | %s (%s, Age %d)\n", p.jerseyNo, p.name, p.position, p.age);
            printf("          Matches: %d | Goals: %d | Assists: %d | G/M: %.2f | Rating: %.2f\n",
                   p.matches, p.goals, p.assists, p.goalsPerMatch, p.rating);
```

```
        }
    } else if (mode == 2) {
        char query[NAME_LEN];
        printf("  Enter Name (or part of name): ");
        fgets(query, NAME_LEN, stdin);
        trimNewline(query);

        int found = 0, i;
        for (i = 0; i < playerCount; i++) {
            if (strstr(squad[i].name, query) != NULL) {
                Player p = squad[i];
                printf("\n  [MATCH] Jersey #%d | %s (%s, Age %d)\n", p.jerseyNo, p.name, p.position, p.age);
                printf("          Matches: %d | Goals: %d | Assists: %d | G/M: %.2f | Rating: %.2f\n",
                       p.matches, p.goals, p.assists, p.goalsPerMatch, p.rating);
                found++;
            }
        }
        if (!found) {
            printf("  [!] No player found matching '%s'.\n", query);
        }
    } else {
        printf("  [!] Invalid search mode selected.\n");
    }
}

 * MODULE 5: UPDATE PLAYER STATS (LOG NEW MATCH PERFORMANCE)
void updatePlayerStats(void) {
    int jNo;
    printf("\n  --- LOG MATCH PERFORMANCE / UPDATE STATS ---\n");
    printf("  Enter Player Jersey Number to update: ");
    if (scanf("%d", &jNo) != 1) {
        clearInputBuffer();
        return;
    }
    clearInputBuffer();

    int idx = findPlayerIndexByJersey(jNo);
    if (idx == -1) {
        printf("  [!] Player with Jersey #%d not found!\n", jNo);
        return;
    }

    int addMatches, addGoals, addAssists;
    printf("  Updating Record for: %s (Current: %d Matches, %d Goals, %d Assists)\n",
           squad[idx].name, squad[idx].matches, squad[idx].goals, squad[idx].assists);
    printf("  Enter Additional Matches Played : ");
    scanf("%d", &addMatches);
    printf("  Enter Additional Goals Scored   : ");
    scanf("%d", &addGoals);
    printf("  Enter Additional Assists        : ");
    scanf("%d", &addAssists);
    clearInputBuffer();

    squad[idx].matches += addMatches;
    squad[idx].goals   += addGoals;
    squad[idx].assists += addAssists;
    recalculateMetrics(&squad[idx]);
    saveToFile();

    printf("\n  [+] Updated! %s now has %d Goals & %d Assists in %d Matches (Rating: %.2f)\n",
           squad[idx].name, squad[idx].goals, squad[idx].assists,
           squad[idx].matches, squad[idx].rating);
}

 * MODULE 6: SQUAD ANALYTICS & GOLDEN BOOT LEADERBOARD (BUBBLE SORT)
void displayTopScorersAndAnalytics(void) {
    if (playerCount == 0) {
        printf("\n  [!] No squad data available for analytics.\n");
        return;
    }

    /* Create a temporary copy of squad array to sort by goals descending */
    Player sorted[MAX_PLAYERS];
    int i, j;
    int totalGoals = 0, totalAssists = 0, totalMatches = 0;
    int bestPlaymakerIdx = 0;

    for (i = 0; i < playerCount; i++) {
        sorted[i] = squad[i];
        totalGoals   += squad[i].goals;
        totalAssists += squad[i].assists;
        totalMatches += squad[i].matches;
        if (squad[i].assists > squad[bestPlaymakerIdx].assists) {
            bestPlaymakerIdx = i;
        }
    }

    /* Bubble Sort in descending order of Goals (Tie-breaker: Assists) */
    for (i = 0; i < playerCount - 1; i++) {
        for (j = 0; j < playerCount - i - 1; j++) {
            if (sorted[j].goals < sorted[j + 1].goals ||
               (sorted[j].goals == sorted[j + 1].goals &&
                sorted[j].assists < sorted[j + 1].assists)) {
                Player temp = sorted[j];
                sorted[j] = sorted[j + 1];
                sorted[j + 1] = temp;
            }
        }
    }

    printf("  Total Squad Size         : %d Players\n", playerCount);
    printf("  Cumulative Squad Goals   : %d Goals (Avg: %.2f per player)\n",
           totalGoals, (float)totalGoals / playerCount);
    printf("  Cumulative Squad Assists : %d Assists\n", totalAssists);
    printf("  Golden Boot Leader       : %s (#%d) - %d Goals (%.2f G/M)\n",
           sorted[0].name, sorted[0].jerseyNo, sorted[0].goals, sorted[0].goalsPerMatch);
    printf("  Top Playmaker (Assists)  : %s (#%d) - %d Assists\n",
           squad[bestPlaymakerIdx].name, squad[bestPlaymakerIdx].jerseyNo,
           squad[bestPlaymakerIdx].assists);
    printf("  ------------------------------------------------------------------------\n");
    printf("  Rank | Jersey | Player Name           | Goals | Assists | G+A  | Rating\n");
    printf("  ------------------------------------------------------------------------\n");
```

```
    int limit = (playerCount < 5) ? playerCount : 5;
    for (i = 0; i < limit; i++) {
        printf("   #%d  |  #%-4d | %-22s |  %-4d |  %-6d | %-4d |  %.2f\n",
               i + 1,
               sorted[i].jerseyNo,
               sorted[i].name,
               sorted[i].goals,
               sorted[i].assists,
               sorted[i].goals + sorted[i].assists,
               sorted[i].rating);
    }
}

 * MODULE 7: DELETE / RELEASE PLAYER FROM SQUAD
void deletePlayer(void) {
    int jNo;
    printf("\n  --- RELEASE / DELETE PLAYER FROM SQUAD ---\n");
    printf("  Enter Jersey Number of player to remove: ");
    if (scanf("%d", &jNo) != 1) {
        clearInputBuffer();
        return;
    }
    clearInputBuffer();

    int idx = findPlayerIndexByJersey(jNo);
    if (idx == -1) {
        printf("  [!] Player with Jersey #%d not found.\n", jNo);
        return;
    }

    char removedName[NAME_LEN];
    strcpy(removedName, squad[idx].name);

    int i;
    for (i = idx; i < playerCount - 1; i++) {
        squad[i] = squad[i + 1];
    }
    playerCount--;
    saveToFile();

    printf("  [+] Player '%s' (Jersey #%d) has been released and file updated.\n",
           removedName, jNo);
}
```

Chapter 7: Experimental Execution & Captured Console Output Screens

The source file `football_squad_manager.c` was compiled using the GNU C Compiler (`gcc`) and executed in an interactive terminal session. The captured output below demonstrates viewing the 8-player starter squad (Option 2), registering **Lamine Yamal (#19)** (Option 1), logging a 2-match, 3-goal, 1-assist update for **Lionel Messi (#10)** (Option 4), generating the **Golden Boot & Playmaker Leaderboard** (Option 5), searching by Jersey Number (Option 3), and saving all 9 records to `squad_records.txt` (Option 7):

```
================================================================================
               FOOTBALL PLAYER STATS & SQUAD MANAGER (v1.0)
          Chandigarh University | BCA (Data Science) | 26CAH-101
================================================================================
  1. Add New Player to Squad
  2. View Complete Squad & Statistics Table
  3. Search Player (By Jersey No. or Name)
  4. Log Match Performance (Update Stats)
  5. Squad Analytics & Golden Boot Leaderboard
  6. Release / Delete Player from Squad
  7. Save Records & Exit System
================================================================================
  Enter your choice (1-7): 2

  --- COMPLETE SQUAD ROSTER & PERFORMANCE TABLE (8 Players) ---
  +--------+------------------------+--------------+-----+-----+-----+-----+-------+--------+
  | Jersey | Player Name            | Position     | Age | Mat | Gls | Ast | G/M   | Rating |
  +--------+------------------------+--------------+-----+-----+-----+-----+-------+--------+
  | #10    | Lionel Messi           | Forward      | 36  | 42  | 34  | 19  | 0.81  | 9.90   |
  | #7     | Cristiano Ronaldo      | Forward      | 38  | 45  | 38  | 11  | 0.84  | 9.90   |
  | #11    | Sunil Chhetri          | Forward      | 39  | 38  | 24  | 9   | 0.63  | 9.30   |
  | #9     | Kylian Mbappe          | Forward      | 25  | 44  | 39  | 10  | 0.89  | 9.90   |
  | #17    | Kevin De Bruyne        | Midfielder   | 32  | 36  | 11  | 26  | 0.31  | 9.18   |
  | #8     | Jude Bellingham        | Midfielder   | 21  | 40  | 21  | 13  | 0.53  | 9.02   |
  | #4     | Virgil van Dijk        | Defender     | 32  | 41  | 5   | 3   | 0.12  | 6.70   |
  | #1     | Thibaut Courtois       | Goalkeeper   | 31  | 35  | 0   | 1   | 0.00  | 6.06   |
  +--------+------------------------+--------------+-----+-----+-----+-----+-------+--------+

  Enter your choice (1-7): 1
  --- ADD NEW PLAYER TO SQUAD ---
  Enter Jersey Number (1-99): 19
  Enter Player Full Name    : Lamine Yamal
  Enter Position            : Forward
  Enter Player Age          : 17
  Enter Matches Played      : 38
  Enter Goals Scored        : 14
  Enter Assists Provided    : 17
  [+] Player 'Lamine Yamal' (Jersey #19) added! Rating: 8.61 | Goals/Match: 0.37

  Enter your choice (1-7): 4
  --- LOG MATCH PERFORMANCE / UPDATE STATS ---
  Enter Player Jersey Number to update: 10
  Updating Record for: Lionel Messi (Current: 42 Matches, 34 Goals, 19 Assists)
  Enter Additional Matches Played : 2
  Enter Additional Goals Scored   : 3
  Enter Additional Assists        : 1
  [+] Updated! Lionel Messi now has 37 Goals & 20 Assists in 44 Matches (Rating: 9.90)

  Enter your choice (1-7): 5
  ================ SQUAD ANALYTICS & GOLDEN BOOT LEADERBOARD ================
  Total Squad Size         : 9 Players
```

```
Cumulative Squad Goals   : 189 Goals (Avg: 21.00 per player)
Cumulative Squad Assists : 110 Assists
Golden Boot Leader       : Kylian Mbappe (#9) - 39 Goals (0.89 G/M)
Top Playmaker (Assists)  : Kevin De Bruyne (#17) - 26 Assists
--------------------------------------------------------------------------
Rank | Jersey | Player Name            | Goals | Assists | G+A  | Rating
--------------------------------------------------------------------------
 #1  |  #9    | Kylian Mbappe          |  39   |  10     | 49   |  9.90
 #2  |  #7    | Cristiano Ronaldo      |  38   |  11     | 49   |  9.90
 #3  |  #10   | Lionel Messi           |  37   |  20     | 57   |  9.90
 #4  |  #11   | Sunil Chhetri          |  24   |  9      | 33   |  9.30
 #5  |  #8    | Jude Bellingham        |  21   |  13     | 34   |  9.02
==========================================================================

Enter your choice (1-7): 3
--- SEARCH PLAYER RECORD ---
Select search mode (1=Jersey, 2=Name): 1  |  Enter Jersey Number: 19
[FOUND] Jersey #19 | Lamine Yamal (Forward, Age 17)
        Matches: 38 | Goals: 14 | Assists: 17 | G/M: 0.37 | Rating: 8.61

Enter your choice (1-7): 7
[+] All squad records saved to 'squad_records.txt'.
[+] Thank you for using Football Player Stats & Squad Manager!
```

Chapter 8: Software Testing Matrix, Conclusion & Bibliography

8.1 Verification & Boundary Test Case Matrix

Test ID	Module Tested	Test Input / Scenario	Expected System Behavior	Status
TC-01	First-Run Auto-Seeding	Launch binary when `squad_records.txt` is absent	Seeds 8 default footballers, computes ratings, and creates `squad_records.txt`	PASS
TC-02	Add Player (Valid)	Jersey `#19`, `Lamine Yamal`, Forward, Age 17, 38 Mat, 14 Gls, 17 Ast	Appends player, computes G/M = 0.37, Rating = 8.61, saves to file	PASS
TC-03	Duplicate Jersey Guard	Attempt to add new player with existing Jersey `#10`	Prints `[!] Error: Jersey #10 is already assigned` and rejects entry	PASS
TC-04	Zero-Appearance Guard	Add youth goalkeeper with `0` matches, `0` goals, `0` assists	Prevents division-by-zero; assigns G/M = 0.00 and baseline Rating = 6.00	PASS
TC-05	Substring Name Search	Select Search Mode 2 and enter partial query `"Chhetri"`	Matches `Sunil Chhetri (#11)` via `strstr()` and prints full profile	PASS
TC-06	Match Performance Update	Update Jersey `#10` with +2 Matches, +3 Goals, +1 Assist	Updates Messi's totals to 44 Matches, 37 Goals, 20 Assists (G/M = 0.84)	PASS
TC-07	Leaderboard Tie-Breaker	Run Option 5 when two players have identical goal counts	Bubble Sort ranks player with higher assist total ahead on leaderboard	PASS
TC-08	Non-Numeric Menu Input	Type alphabetic string `"abc"` at main menu prompt	`scanf` check catches error, `clearInputBuffer()` flushes `stdin` cleanly	PASS

8.2 Conclusion & Summary of Achievements

The **Football Player Stats & Squad Manager** demonstrates how foundational C programming concepts taught in the first semester of **BCA (Data Science)** at **Chandigarh University** can be combined to build a

reliable, data-driven sports management system. By pairing user-defined structures (`typedef struct Player`) with pointer-based metric computation, dual-key Bubble Sort leaderboard ranking, and pipe-delimited flat-file persistence (`squad_records.txt`), the software eliminates manual arithmetic errors and delivers instantaneous insights into squad scoring efficiency and playmaking performance.

Throughout development and testing under the GNU C Compiler (`gcc`), the modular design ensured clean separation between console input sanitization, mathematical metric derivation, and disk storage operations. All eight boundary and functional verification test cases executed with zero runtime exceptions, confirming that the system handles duplicate shirt numbers, zero-appearance division guards, and multi-word string searches accurately.

8.3 Future Scope & Data Science Extensions

- **Automated CSV Dataset Export:** Extend the file I/O module to export comma-separated values (`.csv`) so historical squad records can be imported directly into Python `pandas`, `matplotlib`, and `scikit-learn` pipelines in upcoming Data Science semesters.
- **Position-Specific Performance Models:** Introduce defensive and goalkeeping attributes (clean sheets, interceptions, saves, and pass completion percentage) to evaluate defenders and goalkeepers with role-adjusted rating formulas.
- **Dynamic Memory & Multi-Club League Engine:** Upgrade the fixed-size structure array to dynamically allocated linked lists (`malloc()` and `free()`) to manage multiple club rosters, transfer windows, and tournament fixture tables.

8.4 Bibliography & Reference Materials

1. Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed., ANSI C). Prentice Hall Software Series.
2. Kanetkar, Y. (2022). *Let Us C: Authentic Guide to C Programming Language* (19th ed.). BPB Publications, New Delhi.
3. Balagurusamy, E. (2019). *Programming in ANSI C* (8th ed.). McGraw Hill Education India.
4. Gottfried, B. S. (2018). *Schaum's Outline of Programming with C* (4th ed.). McGraw Hill Education.
5. GNU Project. (2026). *GCC: The GNU Compiler Collection & Standard C Library Reference Manual*. Available at: https://gcc.gnu.org/onlinedocs/